In [ ]:
```
# Customer Behavior Analysis – Alfido Tech
### Name: Dharan S
### Role: Data Analyst Intern
```

In [ ]:
```
## 1. Data Import & Overview
- Loaded dataset using pandas
- Previewed data using head()
- Checked structure using info()
```

In [5]:
```
import pandas as pd
df = pd.read_csv("ecommerce_customer_data_large.csv")
df.head()
```

Out[5]:

	Customer ID	Purchase Date	Product Category	Product Price	Quantity	Total Purchase Amount	Payment Method	Customer Age	Retu
0	44605	2023-05-03 21:30:02	Home	177	1	2427	PayPal	31	
1	44605	2021-05-16 13:57:44	Electronics	174	3	2448	PayPal	31	
2	44605	2020-07-13 06:16:57	Books	413	1	2345	Credit Card	31	
3	44605	2023-01-17 13:14:36	Electronics	396	3	937	Cash	31	
4	44605	2021-05-01 11:29:27	Books	259	4	2598	PayPal	31	

In [19]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250000 entries, 0 to 249999
Data columns (total 13 columns):
 #   Column                 Non-Null Count   Dtype
---  ------                 --------------   -----
 0   Customer ID            250000 non-null  int64
 1   Purchase Date          250000 non-null  object
 2   Product Category       250000 non-null  object
 3   Product Price          250000 non-null  int64
 4   Quantity               250000 non-null  int64
 5   Total Purchase Amount  250000 non-null  int64
 6   Payment Method         250000 non-null  object
 7   Customer Age           250000 non-null  int64
 8   Returns                202618 non-null  float64
 9   Customer Name          250000 non-null  object
 10  Age                    250000 non-null  int64
 11  Gender                 250000 non-null  object
 12  Churn                  250000 non-null  int64
dtypes: float64(1), int64(7), object(5)
memory usage: 24.8+ MB
```

In [21]:
```
df.isnull().sum()
```

Out[21]:
```
Customer ID                 0
Purchase Date               0
Product Category            0
Product Price               0
Quantity                    0
Total Purchase Amount       0
Payment Method              0
Customer Age                0
Returns                 47382
Customer Name               0
Age                         0
Gender                      0
Churn                       0
dtype: int64
```

In [ ]:
```
## 2. Data Cleaning & Preprocessing
- Handled missing values in Returns column
- Converted data types where necessary
- Removed redundant columns
```

In [25]:
```
df["Returns"] = df["Returns"].fillna(0)
```

In [27]:
```
df.isnull().sum()
```

Out[27]:
```
Customer ID              0
Purchase Date            0
Product Category         0
Product Price            0
Quantity                 0
Total Purchase Amount    0
Payment Method           0
Customer Age             0
Returns                  0
Customer Name            0
Age                      0
Gender                   0
Churn                    0
dtype: int64
```

In [29]:
```
df["Returns"].value_counts()
```

Out[29]:
```
Returns
0.0    148524
1.0    101476
Name: count, dtype: int64
```

In [31]:
```
df["Returns"] = df["Returns"].astype(int)
```

In [33]:
```
df["Returns"].value_counts()
```

Out[33]:
```
Returns
0    148524
1    101476
Name: count, dtype: int64
```

In [35]:
```
df.head()
```

Out[35]:

	Customer ID	Purchase Date	Product Category	Product Price	Quantity	Total Purchase Amount	Payment Method	Customer Age	Retu
0	44605	2023-05-03 21:30:02	Home	177	1	2427	PayPal	31	
1	44605	2021-05-16 13:57:44	Electronics	174	3	2448	PayPal	31	
2	44605	2020-07-13 06:16:57	Books	413	1	2345	Credit Card	31	
3	44605	2023-01-17 13:14:36	Electronics	396	3	937	Cash	31	
4	44605	2021-05-01 11:29:27	Books	259	4	2598	PayPal	31	

In [ ]:

```
## 3. Exploratory Data Analysis (EDA)
- Analyzed summary statistics
- Observed distributions of numerical features
- Identified patterns in purchase behavior
```

In [37]:

```
df.describe()
```

Out[37]:

	Customer ID	Product Price	Quantity	Total Purchase Amount	Customer Age	F
count	250000.000000	250000.000000	250000.000000	250000.000000	250000.000000	250000.
mean	25017.632092	254.742724	3.004936	2725.385196	43.798276	0.
std	14412.515718	141.738104	1.414737	1442.576095	15.364915	0.
min	1.000000	10.000000	1.000000	100.000000	18.000000	0.
25%	12590.000000	132.000000	2.000000	1476.000000	30.000000	0.
50%	25011.000000	255.000000	3.000000	2725.000000	44.000000	0.
75%	37441.250000	377.000000	4.000000	3975.000000	57.000000	1.
max	50000.000000	500.000000	5.000000	5350.000000	70.000000	1.

In [7]:

```
df.drop("Customer Age", axis=1, inplace=True)
```

In [9]:
```
df.duplicated().sum()
```

Out[9]:
```
0
```

In [11]:
```
df["Purchase Date"] = pd.to_datetime(df["Purchase Date"])
```

In [13]:
```
df["Year"] = df["Purchase Date"].dt.year
```

In [15]:
```
df["Month"] = df["Purchase Date"].dt.month
```

In [ ]:
```
## 4. Feature Engineering
- Created Total Spend = Quantity × Product Price
- Extracted Year and Month from Purchase Date
```

In [17]:
```
df["Total Spend"] = df["Quantity"] * df["Product Price"]
```

In [19]:
```
df[["Quantity", "Product Price", "Total Spend"]].head()
```

Out[19]:

	Quantity	Product Price	Total Spend
0	1	177	177
1	3	174	522
2	1	413	413
3	3	396	1188
4	4	259	1036

In [ ]:
```
## 5. Customer-Level Analysis
- Aggregated data by Customer ID
- Calculated total spending per customer
- Analyzed purchase frequency
```

In [21]:
```
df.groupby("Customer ID")["Total Spend"].sum().head()
```

Out[21]:
```
Customer ID
1    5600
2    6459
3    3613
4    4339
5    2263
Name: Total Spend, dtype: int64
```

In [23]:
```
customer_spend = df.groupby("Customer ID")["Total Spend"].sum()
```

In [27]:
```
customer_frequency = df.groupby("Customer ID")["Purchase Date"].count()
```

In [29]:
```
import datetime as dt
```

```
recency = df.groupby("Customer ID")["Purchase Date"].max()
recency = (df["Purchase Date"].max() - recency).dt.days
```

In [ ]:
```
## 6. RFM Segmentation
- Recency: Days since last purchase
- Frequency: Number of purchases
- Monetary: Total spending
- Used these metrics to evaluate customer value
```

In [31]:
```
rfm = pd.DataFrame({
    "Recency": recency,
    "Frequency": customer_frequency,
    "Monetary": customer_spend
})
```

In [33]:
```
rfm.head()
```

Out[33]:

Customer ID	Recency	Frequency	Monetary
1	288	3	5600
2	72	6	6459
3	222	4	3613
4	441	5	4339
5	424	5	2263

In [ ]:
```
## 7. Customer Segmentation
- Assigned R, F, M scores using quartiles
- Combined scores to form RFM Score
- Segmented customers into:
  - Best Customers
  - Loyal Customers
  - At Risk
  - Others
```

In [35]:
```
rfm["R_rank"] = rfm["Recency"].rank(ascending=False)
rfm["F_rank"] = rfm["Frequency"].rank(ascending=True)
rfm["M_rank"] = rfm["Monetary"].rank(ascending=True)
```

In [37]:
```
rfm["R_score"] = pd.qcut(rfm["R_rank"], 4, labels=[4,3,2,1])
rfm["F_score"] = pd.qcut(rfm["F_rank"], 4, labels=[1,2,3,4])
rfm["M_score"] = pd.qcut(rfm["M_rank"], 4, labels=[1,2,3,4])
```

In [39]:
```
rfm["RFM_Score"] = rfm["R_score"].astype(str) + rfm["F_score"].astype(str) + rfm["M
```

In [41]:
```
rfm.head()
```

Out[41]:

Customer ID	Recency	Frequency	Monetary	R_rank	F_rank	M_rank	R_score	F_score	M_
1	288	3	5600	16869.0	9331.0	39874.0	3	1	
2	72	6	6459	37913.5	34187.5	43682.0	1	3	
3	222	4	3613	21640.5	17245.5	25097.0	3	2	
4	441	5	4339	9481.0	26093.5	31436.0	4	2	
5	424	5	2263	10114.0	26093.5	12335.5	4	2	

In [43]:

```
rfm["Segment"] = "Others"

rfm.loc[(rfm["R_score"]==4) & (rfm["F_score"]>=3) & (rfm["M_score"]>=3), "Segment"]

rfm.loc[(rfm["R_score"]>=3) & (rfm["F_score"]>=3), "Segment"] = "Loyal Customers"

rfm.loc[(rfm["R_score"]<=2) & (rfm["F_score"]<=2), "Segment"] = "At Risk"
```

In [45]:

```
rfm["Segment"].value_counts()
```

Out[45]:

```
Segment
At Risk            24805
Loyal Customers    17120
Others              6091
Best Customers      1645
Name: count, dtype: int64
```

In [ ]:

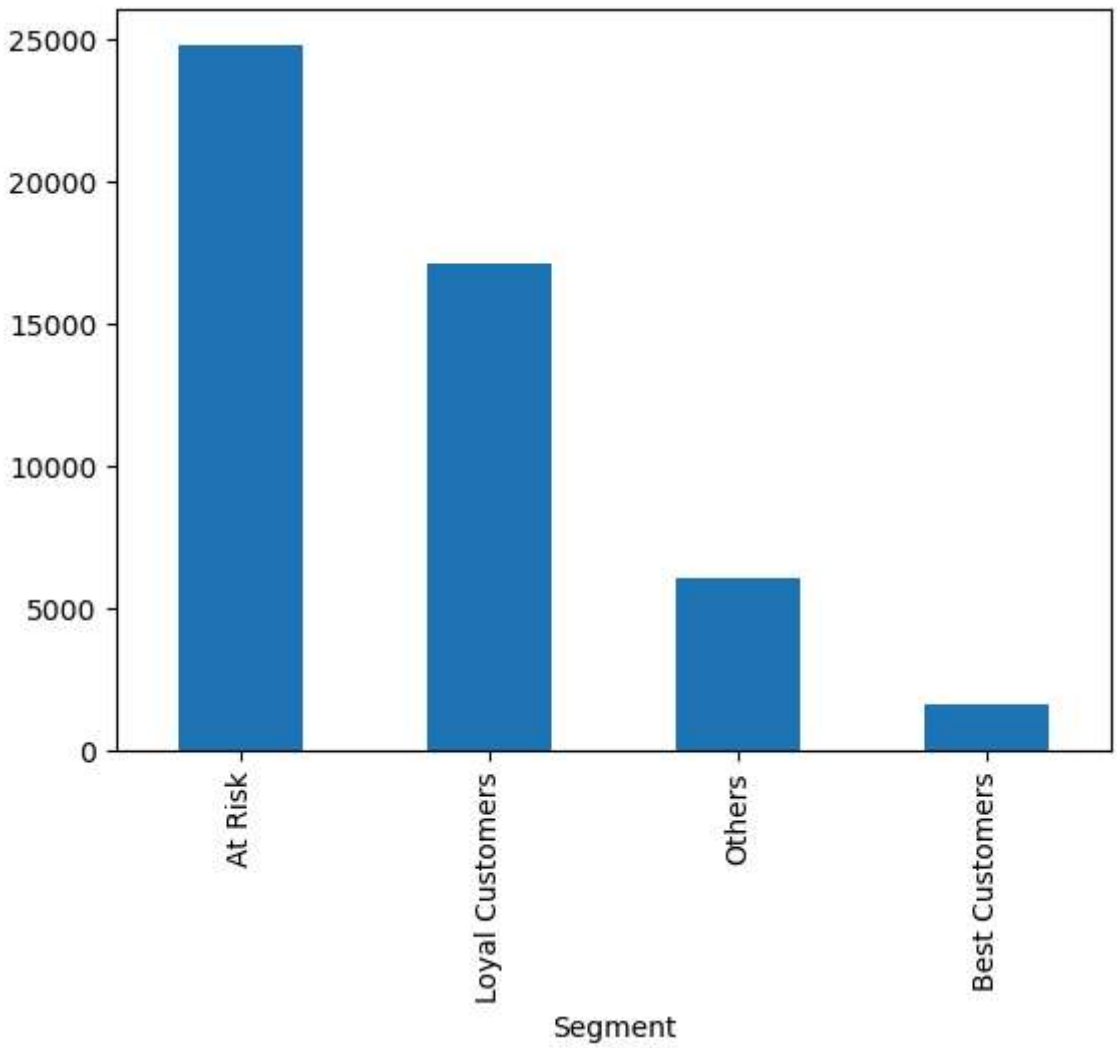
```
## 8. Data Visualization
- Visualized customer segments
- Analyzed product category distribution
- Studied payment method usage
- Explored churn distribution
```

In [47]:

```
rfm["Segment"].value_counts().plot(kind="bar")
```

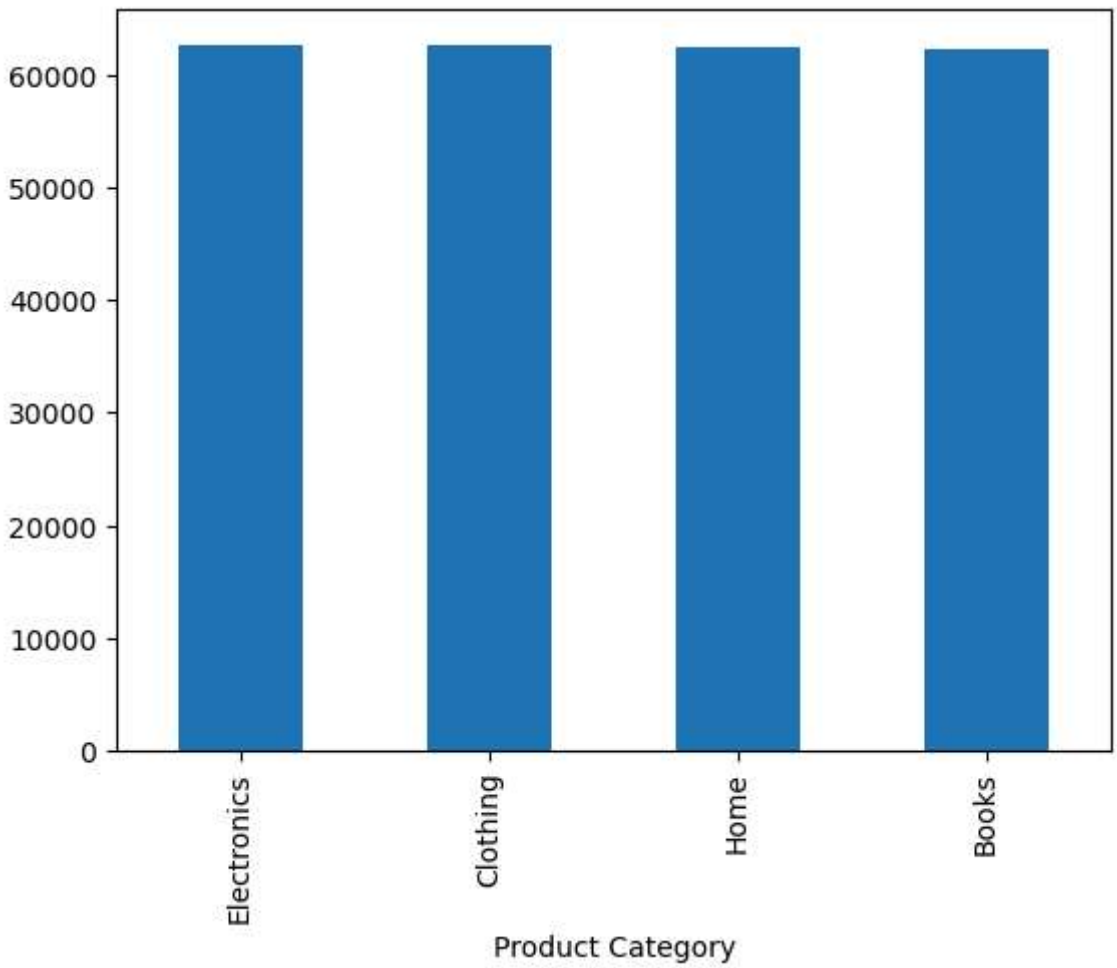
Out[47]:

```
<Axes: xlabel='Segment'>
```



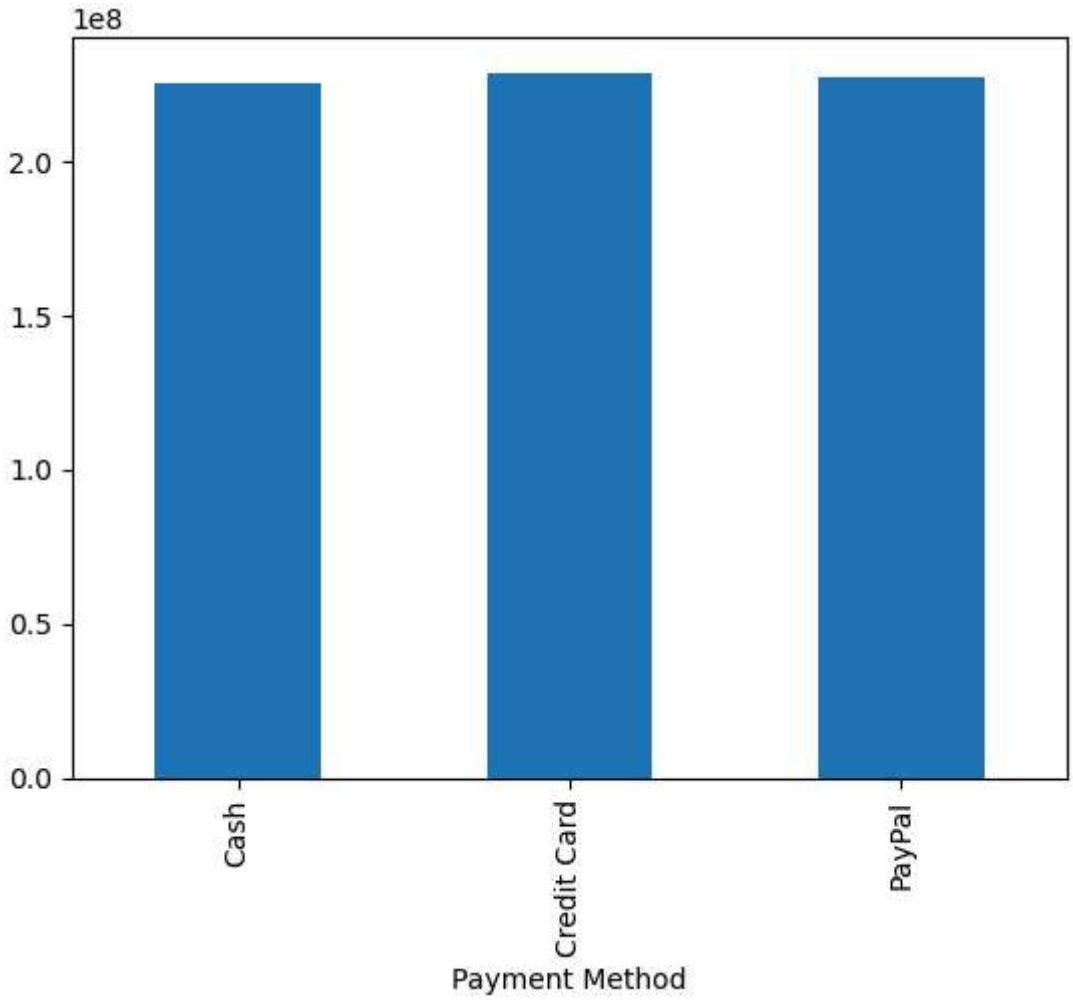
In [49]:
```
df["Product Category"].value_counts().plot(kind="bar")
```

Out[49]:
```
<Axes: xlabel='Product Category'>
```



In [51]:
```
df.groupby("Payment Method")["Total Purchase Amount"].sum().plot(kind="bar")
```

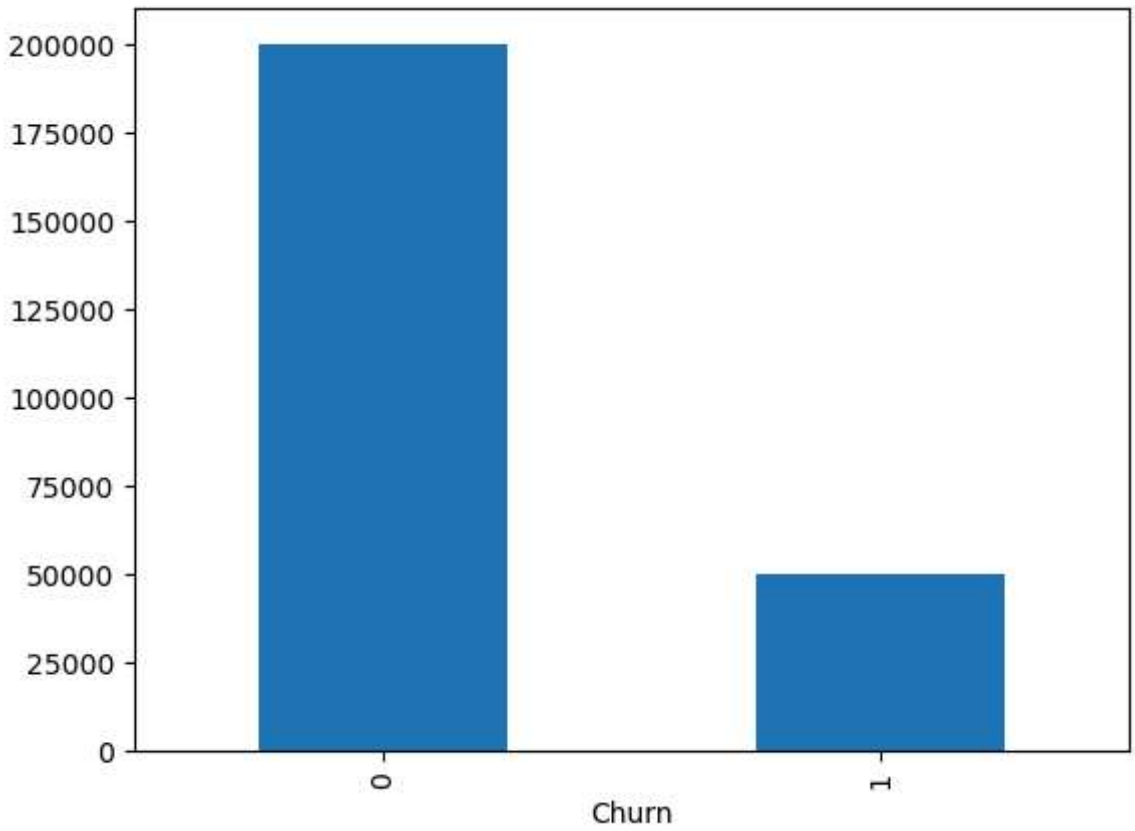
Out[51]:
```
<Axes: xlabel='Payment Method'>
```



In [ ]:
```
## 9. Churn Analysis
- Calculated churn rate across segments
- Compared customer groups
- Identified retention patterns
```

In [53]:
```
df["Churn"].value_counts().plot(kind="bar")
```

Out[53]:
```
<Axes: xlabel='Churn'>
```



In [59]:
```
rfm.groupby("Segment")["RFM_Score"].count()
```

Out[59]:
```
Segment
At Risk            24805
Best Customers      1645
Loyal Customers    17120
Others              6091
Name: RFM_Score, dtype: int64
```

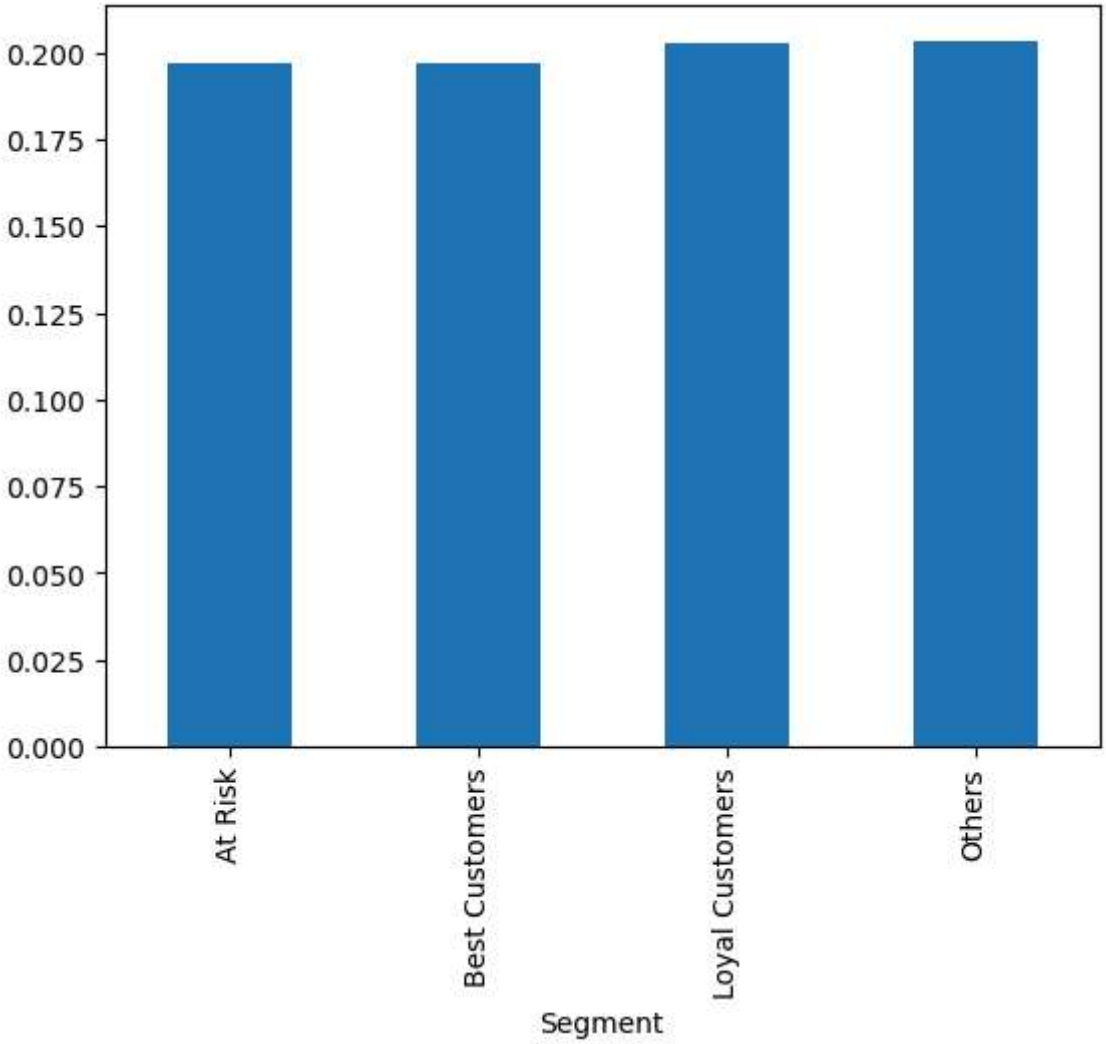
In [63]:
```
df = df.merge(rfm[["Segment"]], on="Customer ID", how="left")
```

In [65]:
```
df.groupby("Segment")["Churn"].mean()
```

Out[65]:
```
Segment
At Risk            0.197005
Best Customers     0.196887
Loyal Customers    0.202768
Others             0.203235
Name: Churn, dtype: float64
```

In [67]:
```
df.groupby("Segment")["Churn"].mean().plot(kind="bar")
```

Out[67]:
```
<Axes: xlabel='Segment'>
```



In [ ]:
```
## 10. Key Insights & Recommendations

### Key Insights:
- Majority customers fall under the "At Risk" segment
- Best Customers are very few
- Loyal Customers form a strong base
- Churn rate is ~20% across all segments
- No segment shows significantly lower churn

### Recommendations:
1. Implement retention campaigns for At Risk customers
2. Reward loyal customers with loyalty programs
3. Provide exclusive offers to Best Customers
4. Use personalized marketing strategies
5. Improve engagement through targeted promotions
```